Intro

[Design Question] Cart

How to read this document?

The question is divided into milestones, each milestone representing a specific product requirement(s).

At the beginning of each milestone, the question is presented in *italics*.

Each milestone is further broken down into one or more steps, which include a suggested solution, key success factors, and considerations to keep in mind.

Continue navigating this doc using the document tabs.

Intro

Our goal is to design an "eCommerce Cart and Checkout" functionality for Wix sites.

A user can navigate to a Wix site that is selling various products, add them to a cart, and complete their purchases using a checkout flow.

For example: The Wix site "Wix n' Mix Marketplace" is using our functionality to sell various products such as: Clothes, jewelry and art items.

Imagine Wix as a small startup where high traffic isn't a primary concern at this stage; the immediate focus is to launch this feature quickly.

MS1: Basic flow

Milestone #1: Basic flow

Our current system supports the following API that is given out of the box:

payOrder(items[], orderInfo) *inserts a new Order to the database - returns* orderId *upon successful completion of payment.*

- item - *A single product. For example, a yellow shirt is an item. This is your item catalog.*
- orderInfo - *the payment method, shipping details, etc.*

We want to extend our system to support cart functionality. We want to implement the following APIs:

- addCartItem(itemId, siteId, userId)
- getCartItems(siteId, userId)
- checkoutCart(siteId, userId, orderInfo)
    - *should use our existing* payOrder *API*

siteId & userId - *the Wix site and the Wix user*

Interviewer Notes/Hidden requirements

- Entities that are given when the candidate asks:
    - `Items` (Catalog): An Item is immutable and cannot change.
    - `Order`: Represents a group of items we successfully paid for.
    - `User & Site`: The logged-in Wix user and the Wix site.
- No need to manage inventory - Assume unlimited stock for all items.
- A user has one active cart per site; the cart clears after checkout.

- Authentication and Authorization - out of scope.
    - If anyone asks why userId/siteId is passed inline (and not in a cookie) - tell them it's actually injected from the cookie. Doesn't really matter for the purpose of our question.
- If the candidate asks about the `Item` or `Order` Entity and what they are, you can share their entities <u>but this is not really needed for the design</u>:
    - `Order`: `order_id, items, order_info, amount, status`
    - `Item`: `item_id, Site_id, name, price`
- `order_id` - should be saved somewhere to connect a `Cart` to a payment to keep track of our payment.

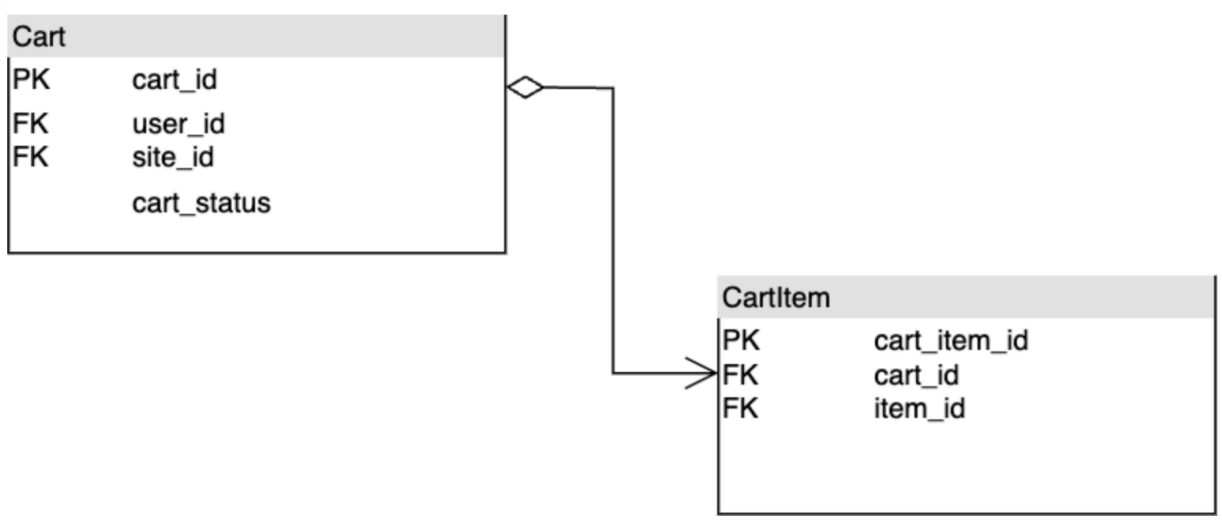
  When to move to Milestone #2 : Complete MS1 & move to MS2 once you have the basic flow working. Failure points discussions can be deferred to the next milestone.

Suggested Solution

Step 1A : Modeling & Databases

<u>Data Modeling</u>:

A possible approach:



Other solutions are valid as well as long as they make sense.

Having only a `CartItem` table with `site_id`, `user_id`, and `item_id`—without a `Cart` table—can be awkward. Ask how will they clear the cart after checkout? Where will they store the `order_id`?

Note that if the candidate suggests modeling the `CartItem` inside `Cart` (e.g. `Cart` has `` `cartItems` `` properties) you can discuss possible race conditions, where multiple writes are editing the same cart.

Database Choice: If their primary expertise lies in NoSQL (or RDBMS), that's perfectly fine. Can they discuss the trade-offs between NoSQL and RDBMS options?

This discussion could be a good opportunity to explore concepts like ACID properties and the CAP Theorem, or you can delve deeper into those later.

Primary Key Design: How did they decide to structure primary keys?

Did they consider using *UUIDs* versus *auto-incremented counters*? Can they discuss the tradeoffs of each approach?

Indexing: Which columns are suitable for a database index? Explain your reasoning. What are the implications of creating an index for every column combination?

Step 1B : End-to-end flow

Network protocols & REST: Can they explain what REST is and why it's commonly used? What is the underlying network protocol for REST? Any other network protocols they know?

Features Implementation:
The candidate should explain how to implement the following (verbal end-to-end explanation or very short pseudo code)

- addCartItem(itemId, siteId, userId)
    - Fetch the active `Cart` by our siteId & userId. If there's no active `Cart`, create a new `Cart` with an "`ACTIVE`" status.
    - Add an entry in the `CartItem` table to associate the item with the `Cart`

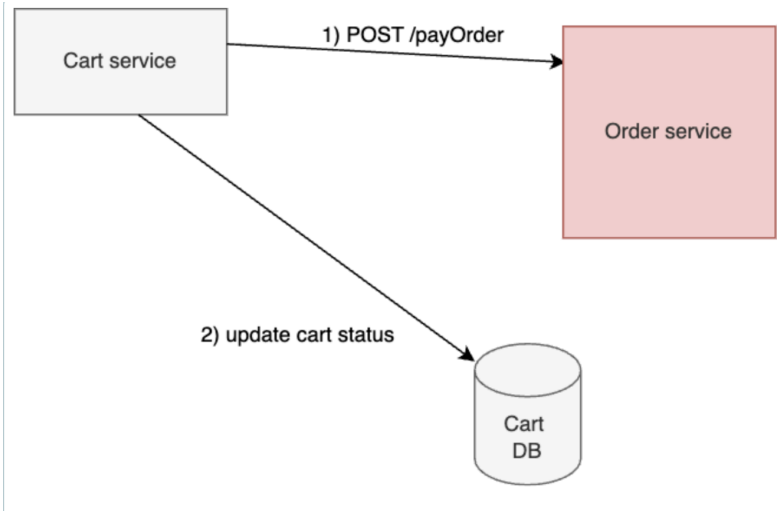
- getCartItems(siteId, userId): Fetches all `CartItems` for an active `Cart` by our siteId & userId.

  Did they mention the use of pagination? How would they go about implementing it?

- checkoutCart(siteId, userId, orderInfo) **:** Fetches all `CartItems` for an active `Cart` by our siteId & userId.

  Then we can call our existing /payOrder(items[], orderInfo) endpoint.

  If the payment is successful, the cart's status is updated to "`COMPLETED`".



  Make sure the candidate understands that a user has one active cart per site and the cart clears after checkout. We can use the cart's status to mark a Cart as "`COMPLETED`" - other approaches might be valid as well.

Evaluation

Less-experienced candidates:

- Proper database modeling.
- Choosing the correct fields to index & PK
- Outlining the flow diagram and thoroughly explaining the read and write flow.
- NoSQL vs RDBMS - basic characteristics. Make sure they have some understanding of the tech stack they worked in.

More-experienced candidates:

- NoSQL vs RDBMS - deep dive. Explain concepts such as ACID, CAP.
- How indexing works (in a high level)
- How do you deal with failure points?
    - Self healing
    - Long running retries

MS2: Scale & Stability

Milestone #2: Support high volume of traffic & Product stability

Our business is booming! We have a lot of new users that are visiting our websites.

The system should support high volumes of concurrent users. (10k RPM)

Which components in our system can break?

Suggested Solution

We expect the candidate to bring up:

- Load balancers
- Scaling up vs scaling out our services
- Database issues - where we would want to especially elaborate about replication.

You can also talk about our App's threading model (e.g. Jetty vs Netty - Request per thread vs asynchronous flow)

We can also ask about more general cross cutting concerns such as: Logging, monitoring, auto-scaling, alerting, over provisioning and analytics

Failure Scenarios: Did the candidate consider failure scenarios?

> What happens if we fail to call `/payOrder`?
>
> What if it succeeds but the system fails to update the cart status to "`COMPLETED`"?

Idempotency: How can we prevent checking out a cart / paying for a cart more than once? It's also a good place to discuss exactly-once/atleast-once

Replication: If the candidate chooses to model the database using an RDBMS the expectation is that they will describe a **leader and multiple followers** architecture with replication between them.

For a NoSQL choice, ask how replication is handled in their selected database and ask how they think it might work in an RDBMS.

If the candidate brings up sharding at this point, ,make sure they understand that you can have replication without sharding, but you can't have sharding without replication (it doesn't make sense to have only a single copy of each shard, no fault tolerance)

So try gently to guide them through this thought process - and once they realize the above, tell them to focus on replication first, and (up to you) defer sharding to MS4.

Eventual Consistency: What new characteristic does the system have now that we're using replication?

We expect the candidate to identify that **reads can now be eventually consistent**, meaning there could be a delay before data written to the leader is reflected on the followers.

Read your own Writes: How would they handle scenarios where stronger consistency is needed, such as "read your own writes" situation? In our application, this can happen when calling the `addCartItem`, and then `getCartItems`.

We expect the candidate to suggest that **reading directly from the leader** in such cases could help mitigate the issues caused by eventual consistency. What are the tradeoffs?

Caching: If the candidate suggests adding a cache layer - ask them what are they caching and what are they trying to solve? Try to dig a bit deeper and tackle points like cache invalidation.

Evaluation

Less-experienced candidates:

Make sure they have good intuition even on subjects they do not know.

- Being able to explain the difference between scale up/scale out
- Are they able to reason about availability, resilience, latency?
- Why do we need load balancers? Being able to describe how they work at a high level?
- Make sure they understand they have multiple instances of the service and the database
- Explain eventual consistency.
- Identify failure points e.g. anything can fail
- How does threading affect the throughput of our service?

<u>More-experienced candidates:</u>

- Thoroughly explain replication models (e.g. leader and multiple followers)
- Read your own writes

MS3: Email Feature

Milestone #3: Email feature

We would like to send a checkout confirmation email with the `orderId` upon a successful checkoutCart*.*

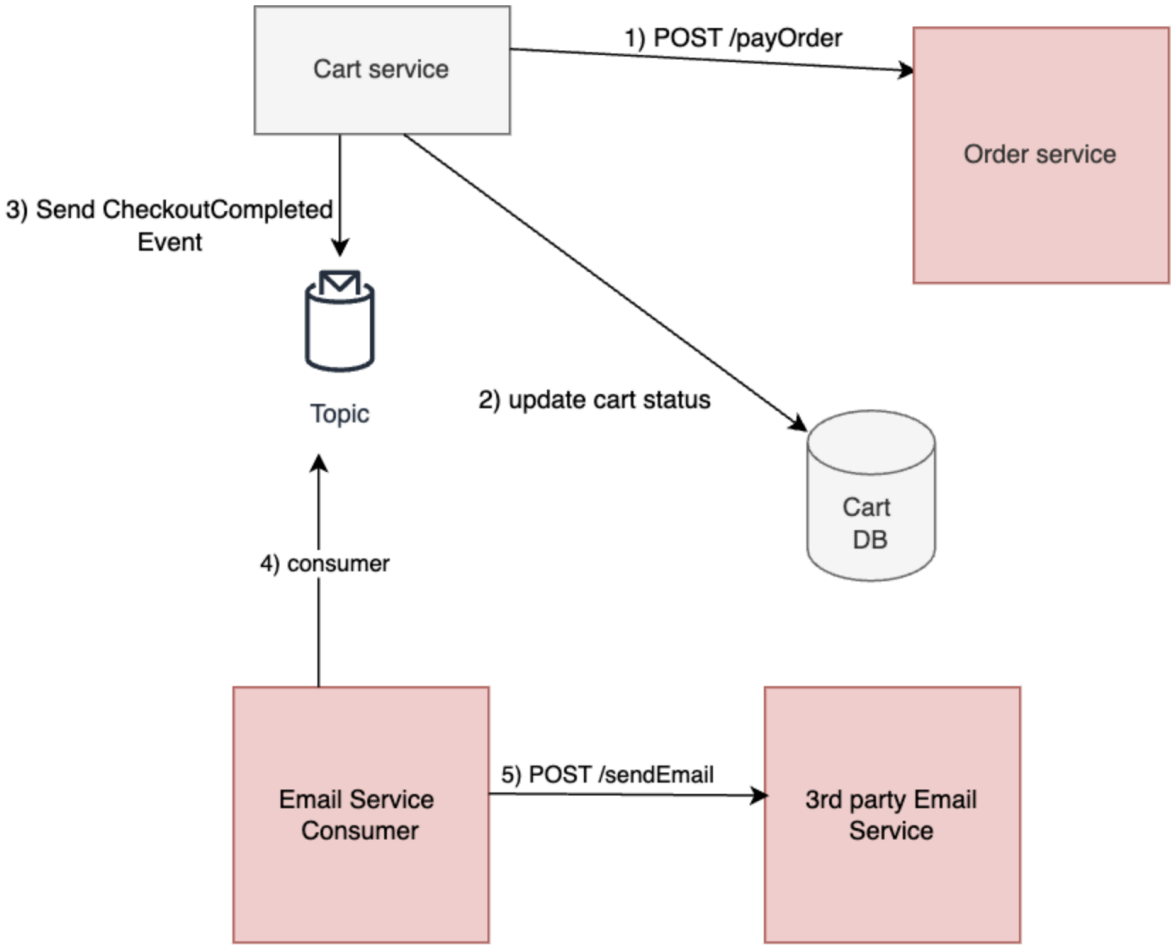
You would need to integrate with an existing third-party service that exposes a `POST /sendEmail` endpoint.

Interviewer Notes

- Email address you get for free (saved on user - not your problem)
- The *orderId* should be persisted next to the cart to be able to send it later

Suggested Solution

Below you will find a possible solution of the flow.



<u>Sync vs Async</u>: Did they choose to call the email service synchronously? If so, what are the downsides of this approach? What are the benefits? Think of a slow email service, an unreliable email service and so on..

> Did the candidate use an asynchronous message broker to handle the email sending process? What message brokers are they familiar with (e.g., Kafka, RabbitMQ, etc.)? This is the time to deep dive into a specific technology.

<u>Event Design</u>: How are they designing the events that are emitted during the process?

> Are they emitting an "`CheckoutCompleted`" event or a "`sendEmailEvent`"? Can they explain the difference between these two types of events? (e.g. In

"`sendEmailEvent`" there's a coupling - The `Cart` service knows an email should be sent)

<u>Orchestration vs. Choreography</u>: Can they distinguish between orchestration and choreography in the context of microservices?

<u>"Atomicity" in distributed systems</u>: Can they guarantee that an email will be sent once we checked out? Can they guarantee that we'll update the cart status once we paid for the order?

At this point you can discuss Saga pattern and two-phase commit and their tradeoffs..

<u>Reliability</u>: Can they propose ways to improve the reliability of the process? (Retries, DLQ)

<u>Evaluation</u>

<u>Less-experienced candidates:</u>

- Explain sync/async flow characteristics
- Make sure they can properly identify critical flows vs background/non-critical flows (e.g. Paying Order vs Email)
- Discuss about retries, DLQ

<u>More-experienced candidates:</u>

- Explain Orchestration vs Choregography tradeoffs
- Understand that we can't achieve atomicity in a distributed transaction when we call multiple services.
    - Saga pattern - how does it work?
    - Two phase commit (2PC) and its cons

MS4: Sharding

Milestone #4: Support large volumes of data

We've addressed the issue of handling concurrent users, but now we're experiencing a significant increase in the number of items we need to manage.

Here are the additional details:

- *100M Wix sites.*
- *10K items per site (in average)*

Interviewer Notes

- The candidate should recognize that it's not feasible to store all this data in a single `Items` table.
- You can remind the candidate that aside from `Cart`, we also have an `Items` table that we got out of the box in milestone #1. Let's use that table to discuss this part of the question.

  As a reminder, the `Item` table has the following columns: `item_id, Site_id, name, price`

Suggested Solution

Choosing the shard key: `site_id` is a good shard key in this scenario.

Did the candidate bring up the concern of hotspots? (e.g. a site is very very active)

Choosing the number of nodes per shard: How do we choose? Instead of trying to accommodate our current needs, we should try to plan a few years ahead in order to avoid frequent rebalancing.

Request routing: How do we route the request to the correct database node?

1) The client can connect to any node. If that node happens to own the partition to which the request applies, it can handle it directly. Otherwise, it routes the request to the correct node.
2) Or we can implement a routing layer that every request will go through.

Rebalancing: Why should we use consistent hashing instead of just doing *hash(key) mod n*?

Migration: How do we migrate data to the new nodes without downtime?

Evaluation

Make sure the candidate does not suggest going for sharding without real justification (e.g. It's a red flag suggesting sharding in milestone #1)

Less-experienced candidates:

- It's okay not to know the term "sharding"
- We expect the understanding that it's impossible to hold the entire data on a single database node, and the intuition of the need to split the data to several nodes.
- Suggest a sharding algorithm: *hash(key) mod n*
- Explore various migration techniques when the number of shards need to change.

More-experienced candidates:

- Can talk about consistent hashing to make rebalancing less painful.